TARGET SHOOTING GAME USING PYTHON TKINTER

AIM

To design and develop an interactive Target Shooting Game using Python Tkinter. A target appears at a random position on the game board. The player clicks the target to earn points and has a limited number of shots. The game displays the score, shots remaining, hit or miss messages, and provides New Game and Exit options.

PROCEDURE

1. Import the tkinter library for the graphical interface and random for target placement.
2. Create the main window and set its title and background colour.
3. Add labels to display the game title, score, shots remaining, and feedback messages.
4. Create a Canvas widget to act as the shooting board.
5. Draw a circular target using concentric circles in different colours.
6. Use the random module to place the target at a random position on the board.
7. Bind the mouse left-click event to the shooting function.
8. When the player clicks, calculate the distance between the click and the target centre.
9. If the click falls inside the target, add 10 points; otherwise, display a miss message.
10. Reduce the remaining shots after each click and update the score and shot labels.
11. When all shots are used, display Game Over and the final score.
12. Provide New Game to reset the score and shots, and Exit to close the application.

PROGRAM CODE

Save this program as **target_shooting_game.py** and run it using Python 3.

```
import tkinter as tk
import random

WIDTH = 700
HEIGHT = 450
TARGET_RADIUS = 28
MAX_SHOTS = 10
TARGET_POINTS = 10

class TargetShootingGame:
    def __init__(self, root):
        self.root = root
        self.root.title("TARGET SHOOTING GAME")
        self.root.resizable(False, False)
        self.root.configure(bg="#172554")

        tk.Label(
            root, text="TARGET SHOOTING GAME",
            font=("Arial Black", 20), bg="#172554", fg="white"
        ).pack(pady=10)

        info = tk.Frame(root, bg="#172554")
        info.pack(fill="x", padx=20)

        self.score_label = tk.Label(
            info, text="SCORE: 0", font=("Arial", 14, "bold"),
            bg="#172554", fg="#FACC15"
        )
        self.score_label.pack(side="left")

        self.shots_label = tk.Label(
            info, text=f"SHOTS LEFT: {MAX_SHOTS}",
            font=("Arial", 14, "bold"), bg="#172554", fg="#BFDBFE"
        )
        self.shots_label.pack(side="right")

        self.canvas = tk.Canvas(
            root, width=WIDTH, height=HEIGHT,
            bg="#DBEAFE", highlightthickness=3,
            highlightbackground="#38BDF8"
        )
        self.canvas.pack(padx=20, pady=12)
        self.canvas.bind("<Button-1>", self.shoot)

        self.message = tk.Label(
            root, text="Click the target to score points!",
            font=("Arial", 11, "bold"), bg="#172554", fg="#86EFAC"
        )
        self.message.pack(pady=3)

        buttons = tk.Frame(root, bg="#172554")
        buttons.pack(pady=8)

        tk.Button(
            buttons, text="NEW GAME", font=("Arial", 11, "bold"),
            bg="#22C55E", fg="black", command=self.new_game
        ).pack(side="left", padx=10)

        tk.Button(
            buttons, text="EXIT", font=("Arial", 11, "bold"),
            bg="#EF4444", fg="white", command=root.destroy
        ).pack(side="left", padx=10)

        self.new_game()

    def new_game(self):
        self.score = 0
        self.shots_left = MAX_SHOTS
        self.game_over = False
        self.score_label.config(text="SCORE: 0")
        self.shots_label.config(text=f"SHOTS LEFT: {MAX_SHOTS}")
        self.message.config(
            text="Click the target to score points!", fg="#86EFAC"
        )
        self.draw_target()

    def draw_target(self):
        self.canvas.delete("all")
        margin = TARGET_RADIUS + 5
        self.target_x = random.randint(margin, WIDTH - margin)
        self.target_y = random.randint(margin, HEIGHT - margin)

        # Draw concentric target rings.
        rings = [
            (TARGET_RADIUS, "#EF4444"),
            (TARGET_RADIUS * 0.75, "white"),
            (TARGET_RADIUS * 0.50, "#EF4444"),
            (TARGET_RADIUS * 0.25, "white"),
            (TARGET_RADIUS * 0.10, "#FACC15")
        ]
        for radius, color in rings:
            self.canvas.create_oval(
                self.target_x - radius, self.target_y - radius,
```

```
                self.target_x + radius, self.target_y + radius,
                fill=color, outline="#111827", width=2
            )

    def shoot(self, event):
        if self.game_over:
            return

        self.shots_left -= 1
        distance = ((event.x - self.target_x) ** 2 +
                    (event.y - self.target_y) ** 2) ** 0.5

        if distance <= TARGET_RADIUS:
            self.score += TARGET_POINTS
            self.score_label.config(text=f"SCORE: {self.score}")
            self.message.config(text="HIT! +10 points", fg="#86EFAC")
        else:
            self.message.config(text="MISS! Try aiming at the target.",
                                fg="#FCA5A5")

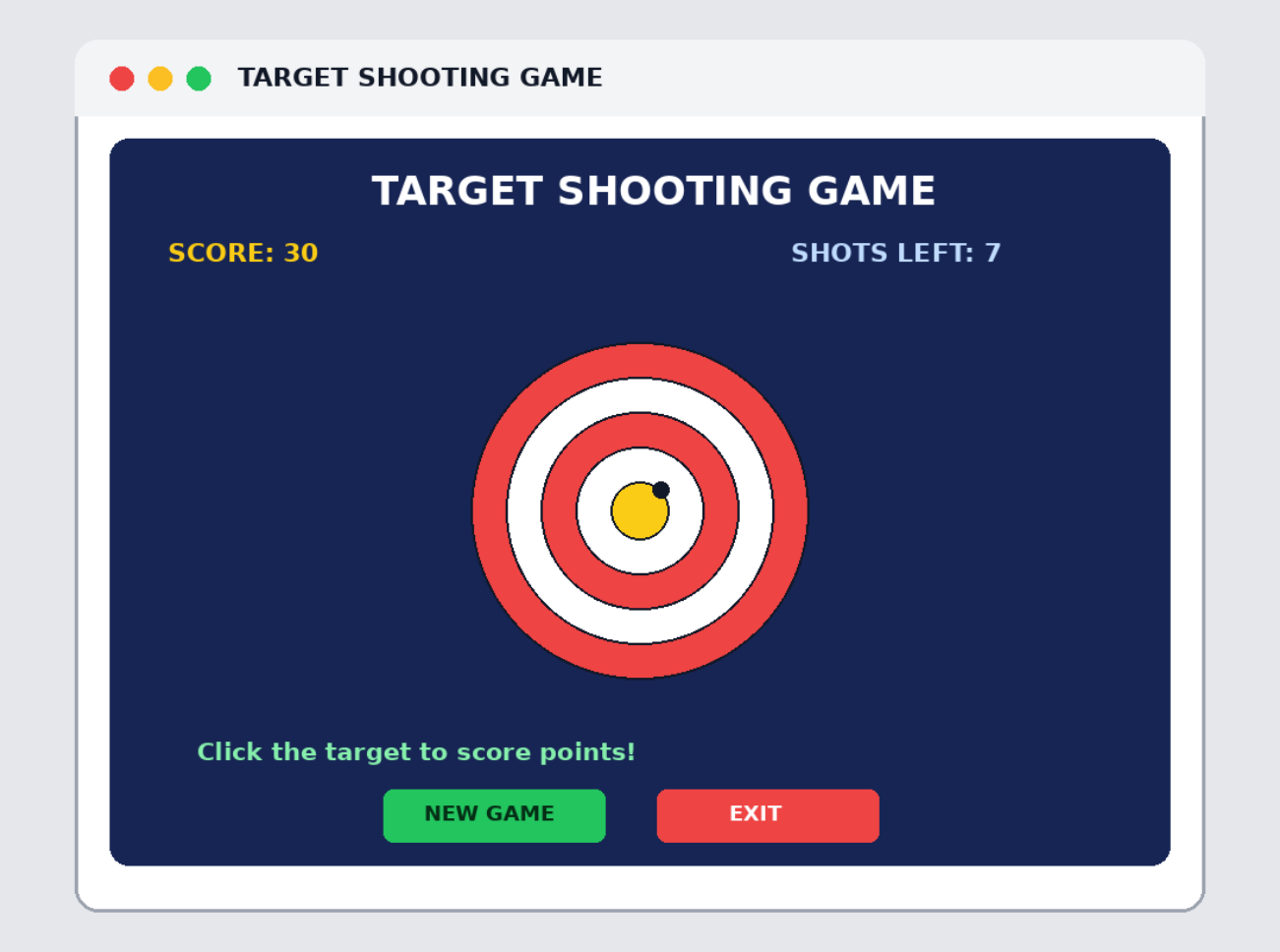
        self.shots_label.config(text=f"SHOTS LEFT: {self.shots_left}")

        if self.shots_left == 0:
            self.game_over = True
            self.message.config(
                text=f"GAME OVER! Final score: {self.score}",
                fg="#FACC15"
            )
        else:
            self.draw_target()

if __name__ == "__main__":
    root = tk.Tk()
    game = TargetShootingGame(root)
    root.mainloop()
```

OUTPUT

The picture below illustrates the target shooting game window. The player clicks the coloured target to score points. The score and shots remaining are shown above the board.



RESULT

Thus, a Target Shooting Game was developed using Python Tkinter. The game demonstrates GUI design, mouse event handling, random target placement, distance calculation, conditional statements, score tracking, and game-state management.